

CAHIER DES CHARGES

Agent de Voyage IA

Application Flask + Serveur MCP + MongoDB

Type de document : Cahier des charges fonctionnel et technique

Date : 22 septembre 2026

Inspiration architecturale : Azure/documentdb-agent-kit

1. Contexte et objectifs

1.1 Contexte

L'objectif est de construire une application web permettant à un utilisateur d'échanger avec un agent IA conversationnel (chatbot) capable de rechercher, comparer et recommander des trajets en avion, en s'appuyant sur une base de données MongoDB contenant les informations de vols (compagnies, aéroports, horaires, prix, disponibilités, escales).

À l'image du projet documentdb-agent-kit, l'architecture repose sur un serveur MCP (Model Context Protocol) qui expose la base de données à l'agent IA de façon contrôlée et sécurisée (outils typés, pas d'accès direct à la chaîne de connexion par l'agent), plutôt que de laisser l'agent générer librement des requêtes MongoDB.

1.2 Objectifs du projet

#	Objectif
O1	Permettre à un utilisateur de rechercher des vols en langage naturel via un chatbot
O2	Exposer la base MongoDB des trajets aériens à l'agent IA via un serveur MCP dédié (recherche, filtrage, comparaison)
O3	Fournir une interface web Flask simple, claire et responsive pour dialoguer avec l'agent
O4	Garantir que l'agent ne peut interroger la base qu'au travers d'outils MCP contrôlés (pas de requêtes arbitraires, pas d'écriture par défaut)
O5	Restituer des réponses structurées (résultats de vols, prix, escales, durée) en plus du texte conversationnel
O6	Poser une base extensible (réservation, alertes de prix, multi-critères) pour des évolutions futures

1.3 Hors périmètre (v1)

- Paiement / réservation réelle de billets
- Intégration à de vraies API de compagnies aériennes (GDS type Amadeus/Sabre) — la v1 utilise une base MongoDB propre, alimentée par des données de démonstration ou un import CSV/JSON
- Authentification multi-utilisateurs avancée (SSO, gestion de rôles fine) — une authentification simple suffit en v1
- Application mobile native

2. Vue d'ensemble de l'architecture

Flux applicatif — de la requête utilisateur à la base de données



Le schéma ci-dessus illustre le flux de bout en bout : la requête de l'utilisateur remonte du navigateur jusqu'à MongoDB en passant successivement par l'application Flask, l'agent IA et le serveur MCP, qui est le seul composant autorisé à interroger la base de données.

2.1 Principe de fonctionnement

1. L'utilisateur saisit une requête en langage naturel dans le chatbot Flask (ex. « Je cherche un vol Paris → Tokyo fin novembre, budget max 700 € »).
2. L'application Flask transmet le message à l'agent IA, qui dispose des outils exposés par le serveur MCP.
3. L'agent décide quel(s) outil(s) MCP appeler (recherche de vols, filtrage par prix/date/escales, détail d'un vol).
4. Le serveur MCP exécute des requêtes MongoDB en lecture seule (par défaut) et retourne des résultats structurés (JSON).
5. L'agent synthétise la réponse en langage naturel ; Flask l'affiche dans le chat (texte + carte de résultats).

3. Modèle de données MongoDB

3.1 Base de données

- Nom de la base : travel_agent_db
- Collection principale : flights

3.2 Schéma du document « flights » (champs principaux)

Champ	Description
flight_number	Numéro de vol (ex. AF1234)
airline.code / airline.name	Code et nom de la compagnie aérienne
departure.airport_code / city / country / datetime	Aéroport, ville, pays et horodatage de départ
arrival.airport_code / city / country / datetime	Aéroport, ville, pays et horodatage d'arrivée
duration_minutes	Durée totale du vol en minutes
stops[]	Liste des escales (aéroport, durée d'escale)
price.amount / currency / fare_class	Prix, devise et classe tarifaire
seats_available	Nombre de sièges disponibles
aircraft_type	Type d'appareil
baggage_allowance_kg	Franchise bagage en kg
updated_at	Date de dernière mise à jour de l'enregistrement

3.3 Collections secondaires (optionnelles v1 / v2)

Collection	Usage
airports	Référentiel des aéroports (code IATA, ville, pays, coordonnées)
airlines	Référentiel des compagnies aériennes
conversations	Historique des échanges chatbot par session/utilisateur (v2)
search_alerts	Alertes de prix enregistrées par l'utilisateur (v2)

3.4 Indexation recommandée

- Index composé sur `departure.airport_code`, `arrival.airport_code`, `departure.datetime` (recherche principale)
- Index sur `price.amount` (tri/filtrage par budget)
- Index sur `airline.code`
- Éventuellement un index de recherche full-text sur `airline.name` / villes pour une recherche tolérante aux fautes de frappe

4. Spécifications du serveur MCP

4.1 Rôle

Le serveur MCP est le seul point d'accès de l'agent IA à MongoDB. Il expose des outils typés plutôt qu'un accès libre en requêtes brutes, afin de limiter la surface d'attaque, garantir des résultats structurés et prévisibles, et permettre l'ajout de règles métier (plafond de résultats, tri par défaut).

4.2 Outils MCP à exposer (v1)

Outil	Description	Paramètres	Mode
<code>search_flights</code>	Recherche des vols selon origine, destination, plage de dates, budget max	<code>origin</code> , <code>destination</code> , <code>date_from</code> , <code>date_to</code> , <code>max_price</code> , <code>max_stops</code>	Lecture
<code>get_flight_details</code>	Détail complet d'un vol par son identifiant/numéro	<code>flight_id</code> ou <code>flight_number</code>	Lecture
<code>compare_flights</code>	Compare plusieurs vols (prix, durée, escales)	liste de <code>flight_id</code>	Lecture
<code>list_airports</code>	Recherche d'aéroports par ville/pays	<code>query</code> (texte libre)	Lecture
<code>cheapest_flights</code>	Retourne les N vols les moins chers, dates flexibles	<code>origin</code> , <code>destination</code> , <code>date_from</code> , <code>date_to</code> , <code>limit</code>	Lecture

4.3 Outils différés (v2, désactivés par défaut)

`create_booking_hold` (pré-réservation), `create_price_alert` — outils d'écriture, protégés par un flag explicite (`ENABLE_WRITE_TOOLS=true`), sur le même principe que la gestion des droits observée dans le kit DocumentDB.

4.4 Exigences de sécurité du serveur MCP

- Connexion à MongoDB configurée côté serveur uniquement (variable d'environnement `MONGODB_URI`), jamais transmise ou modifiable par l'agent à l'exécution
- Séparation lecture/écriture : tous les outils v1 sont en lecture seule
- Validation stricte des paramètres d'entrée de chaque outil (types, plages de dates valides, prix positifs)
- Limitation du nombre de résultats retournés par requête (ex. max 20 vols) pour éviter la surcharge de contexte de l'agent
- Journalisation des appels d'outils (audit minimal)

4.5 Technologie

- Langage : Python
- Bibliothèque MCP : SDK officiel « `mcp` » (Python)
- Driver MongoDB : `pymongo`

- Transport : stdio (intégration agent local) et/ou HTTP/SSE selon le client cible

5. Spécifications de l'application Flask

5.1 Fonctionnalités

- Page de chat unique : saisie du message, historique de conversation, affichage structuré des résultats de vols, indicateur de chargement
- Gestion de session (une session = une conversation), stockage en mémoire ou en base pour la v1
- POST /api/chat — reçoit { session_id, message }, retourne { reply, flights: [...] }
- GET /api/health — vérification de l'état de l'application, de MongoDB et du serveur MCP

5.2 Architecture applicative (arborescence indicative)

```
app/  
  __init__.py          # factory Flask  
  routes/  
    chat.py            # route /api/chat  
    health.py          # route /api/health  
  agent/  
    orchestrator.py    # appel agent IA + gestion des outils MCP  
    mcp_client.py      # client MCP côté Flask  
  templates/  
    chat.html  
  static/  
    css/, js/  
  config.py            # variables d'environnement  
  
mcp_server/  
  server.py            # serveur MCP  
  tools/  
    search_flights.py  
    flight_details.py  
    ...  
  db/  
    mongo_client.py  
  
requirements.txt  
.env.example  
README.md
```

5.3 Technologies

- Backend : Python 3.11+, Flask
- Frontend : HTML/CSS/JS simple (Jinja2 + JS pour l'aspect chat), pas de framework front lourd imposé en v1
- Base de données : MongoDB (local via Docker en développement ; Azure Cosmos DB for MongoDB / Azure DocumentDB ou MongoDB Atlas en production)
- Agent IA : modèle de langage avec function calling / tool use branché sur les outils MCP
- Le module orchestrator.py fait le lien entre la requête HTTP Flask, l'appel au LLM et l'exécution des outils MCP

6. Spécifications du chatbot / agent IA

6.1 Comportement attendu

- Comprendre des requêtes en langage naturel (dates relatives, villes, budgets, préférences d'escales)
- Poser une question de clarification si une information essentielle manque plutôt que de deviner
- Utiliser systématiquement les outils MCP pour toute donnée de vol — ne jamais inventer prix, horaires ou disponibilités
- Présenter les résultats de façon lisible, triés selon la demande (prix, durée)
- Gérer le contexte de la conversation (ex. « et un peu moins cher ? » après une première recherche)

6.2 Prompt système (éléments clés à définir)

- Rôle : assistant de recherche de vols
- Instruction explicite d'utiliser les outils MCP pour toute donnée factuelle
- Interdiction de proposer une réservation/paiement (hors périmètre v1)
- Format de réponse attendu (texte + liste structurée de vols)

6.3 Historique et mémoire

- v1 : mémoire de conversation limitée à la session en cours
- v2 (optionnel) : persistance de l'historique en MongoDB (collection conversations) pour reprise de session

7. Exigences non fonctionnelles

Catégorie	Exigence
Performance	Réponse du chatbot en moins de 5 secondes pour une recherche simple (hors latence réseau du LLM)
Sécurité	Aucune chaîne de connexion MongoDB exposée côté client ; variables sensibles en .env, jamais committées
Fiabilité	Les outils MCP doivent gérer proprement les cas « aucun résultat » et les erreurs de connexion MongoDB
Portabilité	Environnement reproductible via Docker Compose (Flask + MongoDB + serveur MCP)
Testabilité	Tests unitaires sur les outils MCP indépendamment de l'agent IA
Journalisation	Logs des requêtes MongoDB exécutées par les outils MCP, à des fins de debug et d'audit
Accessibilité	Interface de chat lisible, contrastes suffisants, navigation au clavier

8. Modélisation des cas d'usage principaux

Cas d'usage	Description
Recherche simple	« Vol Paris → New York le 15 décembre » → résultats triés par prix
Recherche flexible	« Vol pour Lisbonne pas cher courant octobre » → recherche sur plage de dates, tri par prix croissant
Filtrage par escales	« Je veux un vol direct » → filtre max_stops = 0
Comparaison	« Compare le vol AF1234 et le vol LH5678 » → appel de compare_flights

Cas d'usage	Description
Détail d'un vol	« Donne-moi plus de détails sur le deuxième résultat » → appel de <code>get_flight_details</code>
Absence de résultat	Aucun vol trouvé → l'agent propose d'élargir les dates ou le budget

9. Livrables attendus

- Code source de l'application Flask (chat + affichage des résultats)
- Code source du serveur MCP avec ses outils (`search_flights`, `get_flight_details`, `compare_flights`, `list_airports`, `cheapest_flights`)
- Scripts d'initialisation MongoDB (schéma, index) + jeu de données de démonstration
- Fichier `docker-compose.yml` pour lancer MongoDB + serveur MCP + Flask en local
- Documentation : `README.md` (installation, configuration, lancement), description des outils MCP
- Tests unitaires des outils MCP et de la route `/api/chat`
- Fichier `.env.example` listant les variables requises (`MONGODB_URI`, clé API du LLM, `ENABLE_WRITE_TOOLS`, etc.)

10. Planning indicatif

Phase	Contenu	Durée indicative
1. Cadrage & données	Modèle de données MongoDB, jeu de données de démonstration, indexation	3-4 jours
2. Serveur MCP	Développement des outils de lecture, tests unitaires	4-5 jours
3. Agent IA	Intégration LLM + outils MCP, prompt système, gestion du contexte	4-5 jours
4. Application Flask	Routes, interface de chat, affichage structuré des résultats	4-5 jours
5. Intégration & tests	Tests de bout en bout, Docker Compose, documentation	3 jours
6. Recette	Vérification des critères d'acceptation	2 jours

11. Critères d'acceptation

- ☐ L'utilisateur peut obtenir une liste de vols pertinente en formulant une requête en langage naturel.
- ☐ Toute donnée de vol affichée (prix, horaire, escales) provient effectivement de MongoDB via un outil MCP, jamais générée par le LLM.
- ☐ Les outils d'écriture MCP sont désactivés par défaut et nécessitent une activation explicite.
- ☐ L'application fonctionne de bout en bout via `docker-compose up`.
- ☐ Les tests unitaires des outils MCP passent (recherche, filtrage, absence de résultats).
- ☐ Aucune information sensible (URI MongoDB, clé API) n'apparaît dans le code source ou côté client.
- ☐ La documentation permet à un nouveau développeur de lancer le projet en local en moins de 15 minutes.

12. Risques identifiés

Risque	Impact	Mitigation
Hallucination de l'agent sur les données de vol	Information incorrecte affichée à l'utilisateur	Prompt système strict + obligation d'appel d'outil pour toute donnée factuelle
Base de démonstration peu réaliste	Résultats peu convaincants en démo	Générer un jeu de données varié (compagnies, dates, prix)
Latence cumulée (Flask → LLM → MCP → MongoDB)	Expérience utilisateur dégradée	Mise en cache des recherches fréquentes, limitation du nombre de résultats
Sécurité des identifiants MongoDB	Fuite de la base de données	Variables d'environnement, secrets non versionnés, accès en lecture seule par défaut
Dérive de périmètre vers la réservation réelle	Complexité et risques juridiques/paiement	Cadrage explicite du hors-périmètre v1 dans ce document

Ce cahier des charges définit le périmètre v1 d'un agent de voyage conversationnel. Les évolutions (réservation, alertes de prix, multi-utilisateurs, connexion à de vraies API aériennes) sont identifiées comme pistes v2 et devront faire l'objet d'un cahier des charges complémentaire.